

Linear Algebra And Neural Networks

Jack Foreback

1 Introduction and Motivation

It is difficult to overstate the impact that artificial intelligence has had on our society in the short period since its creation. Every time you check the news there seems to be a new revolution in the field, and a new company proclaiming that their AI model will change the world. In 2024 this revolution poised to upend the AI industry was Kolmogorov-Arnold Networks (KANs) [4].

KANs were originally marketed as a complete replacement for neural networks, which are the underlying architecture for most AI applications like ChatGPT/LLMs. At the time of their creation, posts about KANs spread like wildfire across the internet, as people claimed they were the new "neural network killer". However, in the months following their release, KANs seemed to disappear almost entirely from the mainstream.

In this paper we will investigate the linear algebraic underpinnings of neural networks (NNs) and KANs. We will walk through minimal examples to illustrate key ideas, and connect them to some ideas we developed in Linear Algebra. We will examine precisely how these networks are similar, and how they differ. This discussion will lead us to why Kolmogorov-Arnold Networks seemed to disappear two years ago, and where they show up in the modern day.

2 Neural Networks Introduction

For all of their amazing abilities, today's AI models are mostly built on one simple thing: massive amounts of numerical linear algebra. In MTH 406 we studied how functions can be viewed as transformations, and how to map these transformations to matrices. Neural networks operate using a similar idea, but they generally approximate a function by using many, many transformation matrices chained together with a nonlinear activation function between them.

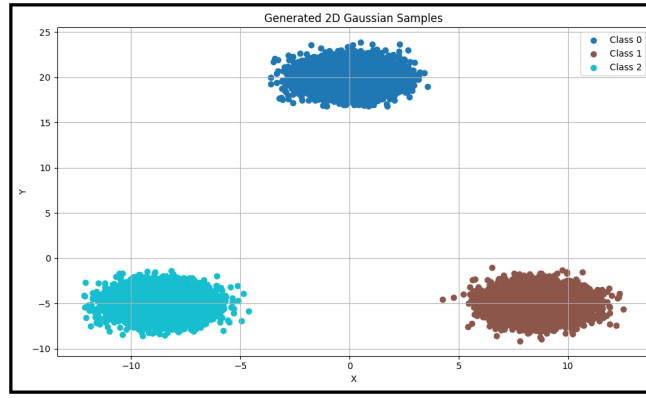
We will now outline a simple example of a neural network, and use it to build an intuitive understanding of the connection to Linear Algebra. This will help us later, as we extend and connect these ideas to Kolmogorov-Arnold Networks.

2.1 Neural Networks Minimal Example

Suppose we have a simple function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ that maps inputs $x \in \mathbb{R}^2$ to outputs $y \in \mathbb{R}^3$ as shown below:

$$Wx + b = y$$
$$\begin{bmatrix} w_{0,0} & w_{0,1} \\ w_{1,0} & w_{1,1} \\ w_{2,0} & w_{2,1} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}.$$

In machine learning, we call this W our "weights matrix". If we were to choose the weights matrix W at random, the function it defines would likely not be very useful, but is there a way to ensure that our matrix defines a useful function? Well, since our inputs are in $\mathbb{R}^2$, we can view them as points in a 2D plane. Suppose we take our points (x_1, x_2) from the three regions shown below:



Now suppose the inputs from each cluster are given unique identifiers, so points from cluster 1 = $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, cluster 2 = $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, and cluster 3 = $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$. Now if we compute $Wx + b = y$ and take the softmax (just scale all entries to sum to 1 and pick the highest) we can interpret this as telling us which cluster of points our weights matrix "predicts" that the input point belongs to! For instance, if our output is $y = \begin{bmatrix} 0.7 \\ 0.2 \\ 0.1 \end{bmatrix}$, this is analogous to our network predicting the input point came from cluster 1 with 70% confidence! This is amazing, because in a few short steps we went from a cluster of points and a random linear transformation to a matrix that can be used to define a decision making function!

I will gloss over the optimization details, but in brief we can train the weights matrix to adapt its entries by feeding in sample points. We update the matrix based on how incorrect its prediction was, typically using the mean squared error. The process of updating the weights matrix using gradients with respect to the parameters is called backpropagation, and it's what actually trains the network to define a weights matrix that "predicts" anything useful.

From this simple example we can see exactly how neural networks are based entirely on linear algebra! We adjust the parameters in our weights matrix to define a function transformation that accomplishes a given task we wish our neural network to execute. You can view the process of giving samples to the network as similar to giving it a point sampled from a function, which it then uses to parameterize and adjust the weights matrix to approximate that function.

I like to think of the mean squared error as making a "line of best fit" for the input data, so that we are approximating towards the least squares solution. You can see how this is all very similar to function interpolation and approximation ideas in numerical linear algebra, but the magic comes from the fact that we can use these ideas to parameterize and "train" our network for ANY function! We can train a function to tie our shoes, solve math problems, or even drive our cars! And that is exactly the connection between numerical linear algebra and AI, at least from a very high level.

We can clearly see how chaining many of these large weights matrices together can produce amazing results, so then why do we need Kolmogorov-Arnold Networks? The answer is that many of our AI models are extremely uninterpretable. They chain together many large weights matrices and nonlinear activation functions, which makes it almost impossible to know exactly what our function is and what our model is doing. KANs offer a more interpretable alternative to neural networks, which is exactly why they are used in fields where interpretability is an advantage.

3 Kolmogorov-Arnold Networks Introduction

KANs work similarly to neural networks, but instead of learning a weights matrix W , they learn functions along their edges directly. KANs learn these functions through summation and composition of basis functions. Recall how in linear algebra a set of linearly independent vectors can be used to form any

vector in their span? You can think of a set of polynomials as being able to form any function similarly, so when we iteratively optimize our network it's similar to finding $c_1, \dots, c_n$ for a linear combination of n basis polynomials.

The power of this comes from the Kolmogorov-Arnold Representation Theorem[2, 7], which says that any multivariate continuous function can be represented exactly with only addition and composition of n inner functions and $2n + 1$ outer functions of a single variable. This can be thought of as having enough basis vectors to "span" your function space. More precisely the theorem says:

Theorem (Kolmogorov, 1957). *For every continuous function $f: [0, 1]^n \rightarrow \mathbb{R}$ there exist continuous functions $\phi_{q,p}: [0, 1] \rightarrow \mathbb{R}$ and functions $\Psi_q: \mathbb{R} \rightarrow \mathbb{R}$ such that:*

$$f(x) = \sum_{q=1}^{2n+1} \Psi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right).$$

To give a general example[5], suppose we have a function $f(x_1, \dots, x_n)$ that we want to represent. First we take $\sum_{p=1}^n \phi_p(x_p) = \phi_1(x_1) + \dots + \phi_p(x_p)$, where ϕ are our continuous "inner" univariate functions. We then pass this through our Ψ function, so $\Psi(\sum_{p=1}^n \phi_p(x_p))$. We can do this $m = 2n + 1$ times to get:

$$\begin{aligned} & \Psi_1(\phi_{1,1}(x_1) + \dots + \phi_{1,n}(x_n)) + \\ & \Psi_2(\phi_{2,1}(x_1) + \dots + \phi_{2,n}(x_n)) + \\ & \dots + \\ & \Psi_m(\phi_{m,1}(x_1) + \dots + \phi_{m,n}(x_n)). \end{aligned}$$

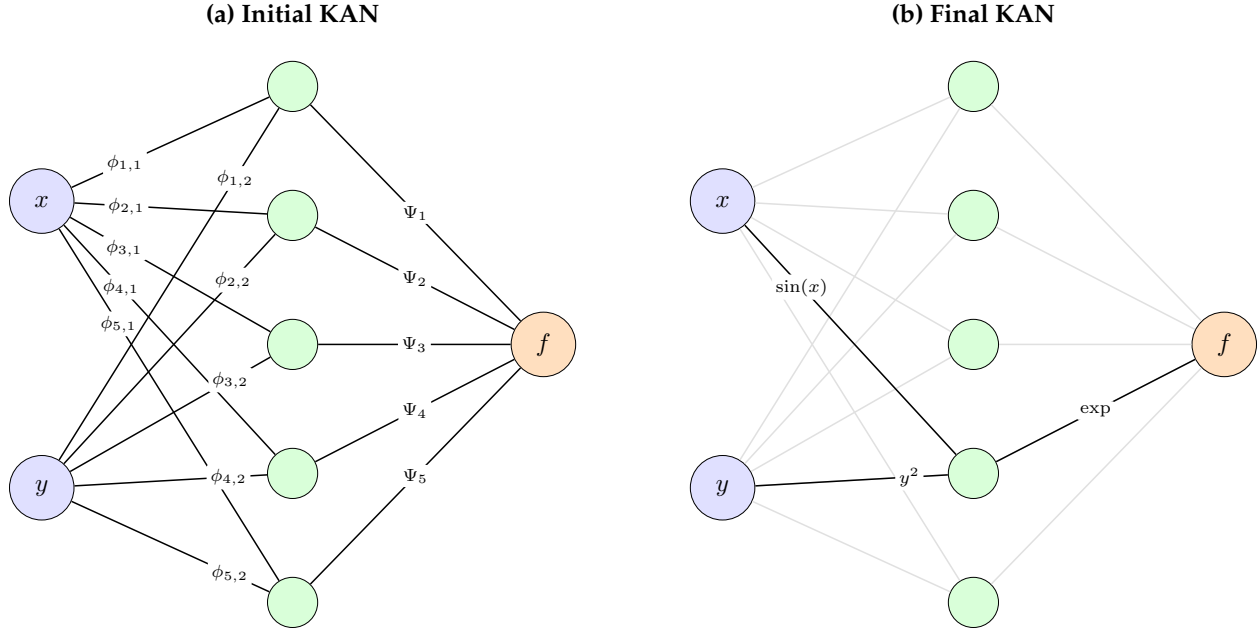
This is exactly our KAN representation from the theorem! We can think of it as a network with shape $[n, 2n + 1, 1]$, where we have n inputs which get mapped by the ϕ functions to a hidden layer of dimension $m = 2n + 1$, which then gets passed through our Ψ functions to produce our output. In matrix form this looks like:

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\text{Input}} \Rightarrow \begin{bmatrix} \phi_{1,1}(x_1) & + \dots + & \phi_{1,n}(x_n) \\ \phi_{2,1}(x_1) & + \dots + & \phi_{2,n}(x_n) \\ \vdots & \vdots & \vdots \\ \phi_{m,1}(x_1) & + \dots + & \phi_{m,n}(x_n) \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{bmatrix} \Rightarrow \underbrace{[\Psi_1(z_1) + \dots + \Psi_m(z_m)]}_{\text{Output}}.$$

We can see the difference from neural networks in that we're no longer doing matrix multiplication, but instead just summing the basis functions evaluated at our input points $x_1, \dots, x_n$. We can think of each $\phi_i(x_j)$ as a scalar for how much each basis function contributes to the linear combination of basis functions that represent the function learned by each edge, or row in our matrix. Now that we've built some mathematical intuition, let's move on to a more visual example.

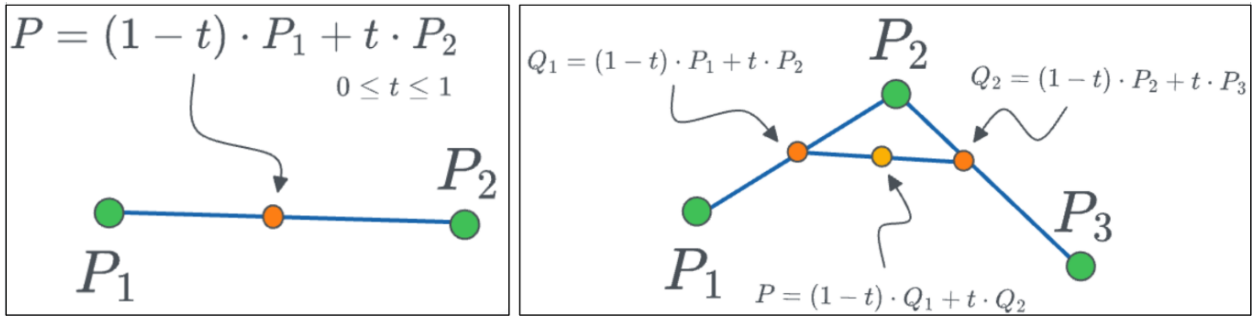
3.1 Kolmogorov-Arnold Networks Example

Let's work through a simple example where $n = 2$ and the function we want to approximate is $f(x, y) = \exp(\sin(\pi x) + y^2)$. This would give our network the shape $[2, 5, 1]$, where we have $n = 2$ input nodes, $2n + 1 = 5$ hidden nodes and 1 output node. There are therefore 10 inner edges, each carrying a learnable function $\phi_{i,j}: \mathbb{R} \rightarrow \mathbb{R}$, and 5 outer edges, each carrying a learnable function $\Psi_i: \mathbb{R} \rightarrow \mathbb{R}$. Our network would look something like the one shown below:



Each hidden node sums the input from every edge, so it adds the values given to it by the activation functions for x and y . We can see this gives us $\sin(x) + y^2$. This is then passed through another edge, which represents another activation function, in this case $\exp = e^x$. This represents composition, as the node value $\sin(x) + y^2$ is passed through the $\exp$ function, or $\exp(\sin(x) + y^2)$. We could also graph these values along each edge.

Now let's dig into how these basis functions actually work to facilitate learning. We will start by discussing Bezier curves, because we talked about them in class and they are quite intuitive. As a quick refresher for Bezier curves, we will follow the example from [1] shown below. If you have two points P_1 and P_2 you can interpolate between them with $P = (1 - t) \cdot P_1 + t \cdot P_2$ (the left figure). Extending this to 3 points gives us two segments, one for P_1 to P_2 and another for P_2 to P_3 . To find the overall Bezier curve, we set our two Bezier segments to variables with $Q_1 = (1 - t) \cdot P_1 + t \cdot P_2$ and $Q_2 = (1 - t) \cdot P_2 + t \cdot P_3$. This allows us to take $P = (1 - t) \cdot Q_1 + t \cdot Q_2$, which is our final result.



This recursive structure gives us a nice formula to find Bezier curves which we can use to make basis functions for our KAN:

$$B(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i P_i.$$

We will continue with Bezier curves for our example, but realize that we can make $B(t)$ any set of basis functions. They can be chosen to be orthogonal, piecewise, or neither, but each come with tradeoffs.

How a KAN actually "learns" is by updating the sum of basis polynomials to fit a desired function. The

true equation is $\phi(x) = w(b(x) + \text{spline}(x))$, but we can ignore w and $b(x)$ for the purpose of clarity, and instead just consider:

$$\phi(x) \approx \text{spline}(x) = \sum_{i=0}^n c_i B_i(x).$$

In the context of Bezier curves our $B_i(t)$ would be $\binom{n}{i}(1-t)^{n-i}t^i$, and our $c_i = P_i$, so it is like we are actually choosing the control points to evaluate our basis polynomials at. Though to be completely honest I don't understand this interpretation intuitively. What makes more sense to me is viewing c_i as a coefficient for each of our B_i basis functions, which when added together for each of our edges defines a function for that edge. Then these learned functions are summed at each hidden node and we learn our Ψ functions to essentially "decode" these hidden nodes back into our desired output. We can train KANs in the same way as neural networks, by using the errors and backpropagation, but in this case our scalars are what we are training for, and if they ever reach all 0 for an edge we can allow this edge to "drop out", so our network becomes smaller and more efficient.

These basis functions are guaranteed to always be able to approximate the function we want them to exactly because of the Kolmogorov-Arnold Representation Theorem, at least in theory. Limitations like computer accuracy and reasonable training time do exist. It can be difficult to visualize KANs, so I had Claude make a tool to do it for me at <https://github.com/JackForeback/KAN-viz>. Now that we hopefully understand most of how KANs work, we can talk about why they disappeared.

4 Why KANs disappeared

From my understanding KANs essentially disappeared for three main reasons. The first is that empirical training was very slow compared to neural networks. This is because matrix multiplication is very fast on modern GPUs, and empirical tests showed that KANs were far slower.

The second is that KANs were essentially shown to be a reformulation of neural networks by an anonymous user on Reddit. They made a colab notebook converting a KAN layer into a neural network layer, which killed all of the hype because it was no longer something "new". I don't fully understand how this works, but it is shown in [6].

Finally, KANs didn't show much promise empirically. They were too expensive and slow to train for any problem at a reasonable scale, so KANs never achieved any comparable results. Training our current AI models cost billions of dollars, so nobody was eager to train a slower, more expensive version.

But there are still niches where KANs are used successfully today. KANs are used as an interpretability tool, and can sometimes outperform standard methods for learning symbolic expressions. Scientific machine learning is also an area where they can be used, because domain knowledge can help improve KAN basis choices[3].

5 Conclusion

That is almost everything I know, and a few things I don't, about Kolmogorov-Arnold Networks, Machine Learning, and Linear Algebra. I likely said a few things incorrectly, but the point of this paper is to express my intuition and understanding, not have every analogy be perfect. There are near infinite resources about linear algebra and deep learning, and it is what many people, including myself, will likely do for their PhD.

After exploring this topic I want to learn more about how to use linear algebra to develop AI assistants and tools to make humanity intelligent enough to solve any problem with ease. After this, I will need to solve the harder problem of keeping these tools out of greedy people's hands. But in all seriousness, this was a great deep dive that made me want to explore machine learning from an entirely mathematical perspective much more often. I will now be looking into some math PhD programs as well as my computer science ones

over the summer to see if I can find anybody doing cool things in the field. Writing and working on this paper made me realize that math and linear algebra for machine learning is much more what I want to do compared to just coding and training big models. This has made a great impact on my future plans, and I'm very happy that I was given the chance to work on this project.

References

- [1] Avi Chawla. A beginner-friendly introduction to Kolmogorov–Arnold networks (KAN). Daily Dose of Data Science, May 2024. Accessed: 2024.
- [2] A. N. Kolmogorov. On the representation of continuous functions of several variables by superpositions of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114:953–956, 1957. English translation: *Amer. Math. Soc. Transl.* (2) **28** (1963), 55–59.
- [3] Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark. KAN 2.0: Kolmogorov–Arnold networks meet science. *Physical Review X*, 2025. Published 17 December 2025; preprint arXiv:2408.10205.
- [4] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov-Arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.
- [5] Andrew Ning. Kolmogorov–Arnold networks (ME 595R: Deep learning for engineers, lecture 19). YouTube video, BYU FLOW Lab, 2026. Brigham Young University. Video available at <https://youtu.be/vAXZi8fPUEA>.
- [6] osamc. Kolmogorov-Arnold network is just an MLP. Reddit post, r/MachineLearning, 2024. Post available at https://www.reddit.com/r/MachineLearning/comments/1clcu5i/d_kolmogorovarnold_network_is_just_an_mlp.
- [7] Wikipedia contributors. Kolmogorov–Arnold representation theorem. Wikipedia, The Free Encyclopedia, 2024. Accessed: 2024.